

TEACHABLE LENS:
A LOCAL FEW-SHOT IMAGE CLASSIFIER USING CLIP AND ONNX RUNTIME

A MINI PROJECT REPORT

Submitted by

SANJAY RAM A

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SSM INSTITUTE OF ENGINEERING AND TECHNOLOGY

DINDIGUL 624 002

ANNA UNIVERSITY::CHENNAI 600 025

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this mini project report "**TEACHABLE LENS: A LOCAL FEW-SHOT IMAGE CLASSIFIER USING CLIP AND ONNX RUNTIME**" is the bonafide work of "**SANJAY RAM A (922123243045)**" who carried out the mini project work under my supervision.

SIGNATURE

Dr. M. PREMKUMAR
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Data Science,
SSM Institute of Engineering and Technology,
Dindigul - 624002.

SIGNATURE

Ms. I. JOICY
ASSISTANT PROFESSOR
(SUPERVISOR)
Department of Artificial Intelligence and Data Science,
SSM Institute of Engineering and Technology,
Dindigul - 624002.

Submitted for the VIVA-VOCE Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Image classifiers are normally trained once for a fixed set of categories, and changing the categories means collecting a new dataset and retraining a deep network, usually on a GPU. This project presents Teachable Lens, a local image-classification application in which the user defines the classes at run time. A pretrained CLIP ViT-B/32 model, exported to ONNX and quantized to 8 bits, is used as a frozen feature extractor that runs on the CPU through ONNX Runtime, without PyTorch or TensorFlow. In few-shot mode, a multinomial logistic-regression head is trained on the 512-dimensional embeddings of 5–20 example photos per class in a few milliseconds. In zero-shot mode, the same encoder matches an image against labels typed as plain text. The system is wrapped in a Gradio browser interface with upload, webcam and clipboard input. On a synthetic shape benchmark the few-shot head reached 87.1% accuracy with one example per class and 100% with twenty, and a leave-one-out test on 20 real photographs of idly and rice gave 100%. An image embedding takes about 23 ms on the CPU and the whole application uses about 475 MB of memory, so it fits comfortably on a laptop with 4 GB of GPU memory and 16 GB of RAM.

LIST OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	**iii**
	LIST OF CONTENTS	**iv**
	LIST OF FIGURES	**vi**
	LIST OF TABLES	**vii**
1	**INTRODUCTION**	**1**
	1.1 SIGNIFICANCE OF ARTIFICIAL INTELLIGENCE IN IMAGE CLASSIFICATION	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 CHARACTERISTICS OF THE PROPOSED SYSTEM	3
	1.5 CHALLENGES IN DESIGNING THE SYSTEM	4
	1.6 MACHINE LEARNING AND TRANSFER LEARNING	4
	1.7 ORGANISATION OF THE REPORT	5
2	**LITERATURE SURVEY**	**6**
	2.1 INTRODUCTION	6
	2.2 LITERATURE REVIEW ON FEW-SHOT AND ZERO-SHOT IMAGE CLASSIFICATION	6
	2.3 SUMMARY OF THE LITERATURE	8
	2.4 RESEARCH GAP	8
	2.5 SUMMARY	9
3	**EXISTING METHODS**	**10**
	3.1 INTRODUCTION	10
	3.2 TRAINING A CONVOLUTIONAL NETWORK FROM SCRATCH	10
	3.3 FINE-TUNING A PRETRAINED NETWORK	10
	3.4 META-LEARNING FOR FEW-SHOT CLASSIFICATION	10
	3.5 NO-CODE TEACHING TOOLS AND CLOUD SERVICES	11
	3.6 ZERO-SHOT CLIP THROUGH FULL DEEP-LEARNING FRAMEWORKS	11
	3.7 LIMITATIONS OF EXISTING METHODS	12
	3.8 SUMMARY	12
4	**PROPOSED SYSTEM**	**13**
	4.1 INTRODUCTION	13
	4.2 SYSTEM ARCHITECTURE	13

CHAPTER NO	TITLE	PAGE NO
	4.3 HARDWARE CONSTRAINTS AND DESIGN DECISIONS	14
	4.4 CLIP VIT-B/32 AS A FROZEN FEATURE EXTRACTOR	14
	4.5 IMAGE PREPROCESSING	14
	4.6 FEW-SHOT MODE: LOGISTIC-REGRESSION HEAD	15
	4.7 ZERO-SHOT MODE: TEXT PROMPTS	16
	4.8 MODELS USED (HUGGING FACE)	16
	4.9 SUMMARY	17
5	**IMPLEMENTATION AND TOOLS**	**18**
	5.1 SOFTWARE ENVIRONMENT	18
	5.2 ONNX RUNTIME	18
	5.3 HUGGING FACE HUB AND TOKENIZERS	18
	5.4 SCIKIT-LEARN AND JOBLIB	19
	5.5 GRADIO	19
	5.6 MODULE STRUCTURE	19
	5.7 EMBEDDING AN IMAGE WITHOUT PYTORCH	19
	5.8 TRAINING THE HEAD	20
	5.9 DATASET STORAGE	20
	5.10 USER INTERFACE	20
	5.11 SUMMARY	21
6	**RESULTS AND DISCUSSIONS**	**22**
	6.1 EVALUATION METHODOLOGY	22
	6.2 EXPERIMENT 1: SYNTHETIC SHAPE BENCHMARK	22
	6.3 EXPERIMENT 2: REAL PHOTOGRAPHS (IDLY VERSUS RICE)	24
	6.4 ZERO-SHOT VERSUS FEW-SHOT CLASSIFICATION	26
	6.5 PERFORMANCE ON THE TARGET HARDWARE	28
	6.6 APPLICATION OUTPUTS	29
	6.7 LIMITATIONS OF THE EVALUATION	30
	6.8 SUMMARY	31
7	**CONCLUSION AND FUTURE WORK**	**32**
	REFERENCES	**34**
	APPENDIX	**36**
	A. SOURCE CODE	36
	B. PER-IMAGE RESULTS	41
	C. HOW TO RUN	41

LIST OF FIGURES

FIGURE NO	FIGURE	PAGE NO
4.1	Proposed system architecture: both modes share the same frozen CLIP encoder	13
4.2	Image preprocessing steps for the CLIP vision encoder (real photograph from the project)	15
6.1	Example images from the synthetic benchmark	23
6.2	Few-shot accuracy versus number of examples per class, with the zero-shot baseline	23
6.3	Confusion matrices on the 90-image test set (zero-shot and few-shot with 10 examples per class)	24
6.4	Leave-one-out predictions on the 20 real photographs (green = correct); #0–9 are rice, #10–19 are idly	25
6.5	Held-out accuracy versus number of training images per class on real photographs	26
6.6	Few-shot (leave-one-out) versus zero-shot accuracy on 20 real photographs	27
6.7	Average CPU latency per item	28
6.8	Teach tab: dataset table and training status	29
6.9	Classify tab: output for an idly photograph	29
6.10	Zero-shot tab: output for a photograph of cooked rice	30

LIST OF TABLES

TABLE NO	TABLE	PAGE NO
2.1	Literature survey of few-shot, zero-shot and CLIP-based classification	8
3.1	Comparison of approaches for building a custom image classifier	12
4.1	Design decisions and their reasons	14
4.2	Pretrained models and files used from the Hugging Face Hub	17
5.1	Libraries used and their versions	18
5.2	Source files and their responsibilities	19
6.1	Few-shot accuracy on the synthetic benchmark (90 test images, 5 random subsets)	23
6.2	Few-shot accuracy on real photographs (idly vs rice), 50 random splits per setting	26
6.3	Zero-shot accuracy on the 20 real photographs for different label wording	27
6.4	Measured performance on the target laptop	28
A.1	Leave-one-out and zero-shot result for each of the 20 real photographs	41

ACKNOWLEDGEMENT

First and foremost, we would like to express our deep sense of gratitude to our most honorable Chairman and Management Trustees **Mr. K. Shanmugavel** for providing us with necessary facilities during the course of study. We feel immensely pleased to express our sincere thanks to our Principal **Dr. S. Karthigai Lakshmi** for encouragement and support extended by them. We extend our solemn gratitude to **Dr. M. Premkumar**, Head of the Department, Artificial Intelligence and Data Science, for timely support to all our activities.

We express our sincere thanks to our guide **Ms. I. Joicy, AP/AI&DS** for her guidance throughout our project work. Also we express our thanks to the faculty members of our department, non-teaching staff members and our dear friends for their moral support, help and encouragement towards the successful completion of the project. We are most indebted to our parents, with whose support, resulted in making our dreams of becoming successful graduates, a reality. We are quite confident that our project work stands testimony to the fact that hard work will bear enjoyable fruits not only to the individuals concerned but to the entire community, as a whole as we have witnessed many inventions of the scientists have made the lives of the brethren more comfortable.

CHAPTER 1
INTRODUCTION

Image classification is the task of assigning a label to a photograph from a set of predefined categories. It sits behind many everyday technologies such as photo search, medical image screening, product quality inspection and crop-disease identification. Most image-classification projects start from a fixed dataset such as CIFAR-10 or MNIST and train a network that recognises exactly those classes. When a user needs a different set of categories, for example their own coffee mug, a particular dish or a hand-drawn logo, the whole cycle of data collection, training and evaluation has to be repeated, and it normally needs programming skills and a GPU.

Teachable Lens takes the opposite direction: the user decides what the classes are at run time. The user creates a class, drops in a handful of example photos, presses Train, and a new classifier is ready in about a second. When there are no examples at all, a zero-shot mode classifies an image against labels typed as plain text. Both modes run fully on the local machine, on the CPU, inside a browser-based interface.

The idea behind the project is transfer learning in its simplest form. A large pretrained vision-language model, OpenAI's CLIP, is used only as a frozen feature extractor. It maps any image to a 512-number vector in which visually and semantically similar images lie close together. A small logistic-regression layer trained on those vectors is enough to separate the user's classes.

1.1 SIGNIFICANCE OF ARTIFICIAL INTELLIGENCE IN IMAGE CLASSIFICATION

Artificial intelligence (AI) is the branch of computer science concerned with building systems that perform tasks which normally require human intelligence, such as perception, reasoning, learning and language understanding. Within AI, computer vision teaches machines to interpret images and video. Deep

neural networks [1] have made computer vision practical at scale: networks trained on millions of labelled images now reach high accuracy on many recognition benchmarks.

Image classification is the most basic vision task and the building block for many others. Typical applications include screening medical scans, sorting produce and manufactured parts, identifying plant diseases from a leaf photograph, organising personal photo libraries, and helping visually impaired people recognise objects around them. In all of these, the categories of interest are specific to the user or the organisation, which makes the ability to define new categories quickly very valuable.

Recent vision-language models change what is possible. By learning from hundreds of millions of image–text pairs, they acquire general visual features that transfer to new tasks with very little additional data [2]. This project builds a complete, usable application around that capability.

1.2 PROBLEM STATEMENT

A conventional image classifier has three practical drawbacks for an ordinary user. First, its label set is fixed at training time, so adding one more category means retraining. Second, training a deep network needs thousands of labelled images per class and, in practice, a GPU together with a heavy framework such as PyTorch or TensorFlow. Third, uploading personal photos to a cloud service to avoid these costs raises privacy concerns.

The target machine for this project is a laptop with 4 GB of GPU memory and 16 GB of system memory. A standard PyTorch installation with CUDA occupies several gigabytes of disk space, and fine-tuning even a small vision transformer within 4 GB of GPU memory is tight. The problem is therefore to build an image classifier that lets a non-expert define arbitrary classes from a few photos, trains in seconds, runs offline on modest hardware and needs no deep-learning framework at run time.

1.3 OBJECTIVES

The objectives of the project are:

- to let non-experts build their own classifier with 5–20 images per class, with no code and no GPU;
- to provide a second, training-free mode that classifies an image against labels typed as text;
- to run fully offline after a one-time model download, on the CPU only;
- to stay within the laptop budget of 4 GB of GPU memory and 16 GB of RAM, without installing PyTorch or TensorFlow;
- to provide a clean browser interface with upload, webcam and clipboard input; and
- to evaluate the accuracy, speed and memory use of the system with reproducible experiments.

1.4 CHARACTERISTICS OF THE PROPOSED SYSTEM

Compared with a typical “train a CNN on a dataset” project, Teachable Lens has the following characteristics:

Run-time classes: there is no fixed label set. Classes are folders of example images that the user can create or delete at any time, and retraining takes milliseconds.

Data efficiency: because the encoder is pretrained, five to twenty photos per class are enough for useful accuracy, as the experiments in Chapter 6 show.

Light weight: the only run-time dependency for the model is ONNX Runtime. The quantized encoders occupy about 147 MB and the whole process needs under 500 MB of memory.

Privacy: images never leave the computer. After the first download of the model files the application works without a network connection.

Two complementary modes: example-based (few-shot) and language-based (zero-shot) classification share a single frozen encoder, so the extra mode costs almost nothing.

Transparent storage: the dataset is an ordinary folder per class, which can be inspected, backed up or edited in a normal file manager.

1.5 CHALLENGES IN DESIGNING THE SYSTEM

Designing a classifier that is accurate, fast and easy to use under the stated constraints involves several challenges.

Limited hardware: the choice of backbone, file format and inference engine must respect 4 GB of GPU memory and 16 GB of RAM. This rules out fine-tuning large networks and favours a quantized model on the CPU.

Very little training data: with only a few images per class, a flexible model overfits easily. Using fixed, general-purpose features and a simple linear classifier keeps the number of trainable parameters small.

Prompt sensitivity in zero-shot mode: the quality of a text-based classifier depends on how the label is worded. Rare or non-English words may not have been seen often during pretraining, as the experiments in Section 6.4 demonstrate.

Quantization loss: 8-bit weights make the files about four times smaller and the CPU inference faster, but they cost a small amount of accuracy compared with the original floating-point model.

Usability and robustness: errors such as training with a single class or uploading a non-image file must be reported as clear messages and never crash the application.

1.6 MACHINE LEARNING AND TRANSFER LEARNING

Machine learning (ML) is a subset of AI in which a program improves at a task by learning from data instead of being explicitly programmed. Machine-learning algorithms fall into three main categories. In *supervised learning* a

model is trained on labelled examples and learns to predict the label of new inputs; image classification is a supervised task. In *unsupervised learning* the model finds structure in unlabelled data, for example by clustering. In *reinforcement learning* an agent learns to take actions that maximise a reward signal.

Transfer learning reuses knowledge learned on one large task for a new, smaller task. In computer vision it is common to take a network pretrained on a large image collection, keep its early layers as a feature extractor and train only a small classifier on top for the new task. This is the approach used here.

Few-shot learning studies how to learn a new category from only a handful of labelled examples. **Zero-shot learning** goes further and recognises categories for which no image examples are available at all, by relating images to a textual description of the class. Vision-language models such as CLIP make both practical because images and text are mapped into the same embedding space.

An **embedding** is a fixed-length vector of numbers that represents an input. In CLIP, the embedding of a photo of a dog lies close to the embedding of the sentence “a photo of a dog” and far from the embedding of “a photo of a car”. The similarity of two embeddings is measured by their cosine similarity.

1.7 ORGANISATION OF THE REPORT

Chapter 1 introduces the problem and the objectives. Chapter 2 surveys the related literature. Chapter 3 reviews existing approaches to building custom image classifiers and their limitations. Chapter 4 describes the proposed system, its architecture and the models used. Chapter 5 explains the tools and the implementation. Chapter 6 presents the experiments, results and screenshots of the application. Chapter 7 concludes the report and lists directions for future work. The source code is reproduced in the Appendix.

CHAPTER 2
LITERATURE SURVEY

2.1 INTRODUCTION

A literature survey identifies the state of the art in a field and the gaps that a new project can fill. This survey covers the research that Teachable Lens builds on: vision transformers, contrastive vision-language pretraining, few-shot learning, the adaptation of CLIP with few examples, existing tools for teaching a classifier without code, and model quantization for efficient deployment.

2.2 LITERATURE REVIEW ON FEW-SHOT AND ZERO-SHOT IMAGE CLASSIFICATION

Vision transformers. Dosovitskiy et al. showed that a standard Transformer, originally designed for text, can be applied directly to images by splitting an image into fixed-size patches and treating the sequence of patch embeddings like a sequence of words [3]. The vision encoder used in this project, ViT-B/32, uses 32×32-pixel patches, so a 224×224 input becomes a sequence of 49 patches plus one summary token.

Contrastive vision-language pretraining. Radford et al. introduced CLIP, which trains an image encoder and a text encoder together on 400 million image–text pairs collected from the internet so that matching pairs have similar embeddings [2]. Because the class names are just text, a trained CLIP model can classify images into any set of categories without further training. The authors reported that zero-shot CLIP matched the accuracy of a supervised ResNet-50 on ImageNet without using any of its 1.28 million labelled training images.

Meta-learning for few-shot classification. Vinyals et al. proposed Matching Networks, which classify a query image by comparing it with a small labelled support set in a learned embedding space [4]. Snell et al. proposed Prototypical Networks, in which each class is represented by the mean embedding of its examples and a query is assigned to the nearest prototype [5]. Finn et al.

proposed MAML, which trains a network so that a few gradient steps on a new task give good performance [6]. These methods are trained on many small episodes drawn from a large labelled dataset.

Strong embeddings are enough. Tian et al. observed that a simple linear classifier trained on top of a good pretrained embedding can match or outperform many sophisticated meta-learning methods on few-shot benchmarks [7]. This finding supports the design choice made here: invest in a strong frozen encoder and keep the trainable part as simple as logistic regression.

Adapting CLIP with few examples. CoOp learns the words of the text prompt from a few labelled images instead of writing them by hand [8]. CLIP-Adapter adds a small trainable feature adapter on top of the frozen encoders [9]. Tip-Adapter builds a cache of the few-shot image features and combines its predictions with those of zero-shot CLIP without any training [10]. Together these works show that a frozen CLIP encoder plus a very small learned component is an effective way to exploit a handful of labelled images.

No-code teaching tools. Carney et al. described Teachable Machine, a web-based tool that lets people train a classifier from webcam examples without writing code [11]. Its aim is the same as that of this project, but it relies on a browser runtime and an image-only feature extractor and has no text-based mode.

Quantization and deployment. Jacob et al. showed that neural networks can run inference with 8-bit integer arithmetic with only a small loss of accuracy while reducing model size and latency [12]. ONNX Runtime executes such quantized models efficiently on ordinary CPUs [13], which makes the approach suitable for a laptop without a deep-learning framework installed.

2.3 SUMMARY OF THE LITERATURE

Table 2.1 summarises the works reviewed and their relevance to this project.

S. No	Reference	Technique	Relevance to this project
1	Dosovitskiy et al., 2021 [3]	Vision Transformer (ViT): image patches processed by a Transformer	Architecture of the CLIP vision encoder (ViT-B/32)
2	Radford et al., 2021 [2]	CLIP: contrastive image–text pretraining on 400 M pairs	Frozen encoder for both modes; zero-shot classification by text prompts
3	Vinyals et al., 2016 [4]	Matching Networks: classify by comparing with a labelled support set	Idea of classifying from a few labelled examples
4	Snell et al., 2017 [5]	Prototypical Networks: class prototype = mean embedding	Embedding-space view of few-shot classes
5	Finn et al., 2017 [6]	MAML: gradient-based meta-learning	Alternative to a frozen encoder; needs training on many tasks
6	Tian et al., 2020 [7]	Linear classifier on a good pretrained embedding	Justifies frozen encoder + logistic-regression head
7	Zhou et al., 2022 [8]	CoOp: learned text-prompt context from few images	Possible future extension of the zero-shot mode
8	Gao et al., 2021 [9]	CLIP-Adapter: small trainable adapter on frozen CLIP	Shows value of a small learned layer on CLIP features
9	Zhang et al., 2022 [10]	Tip-Adapter: training-free cache of few-shot features	Combines few-shot and zero-shot predictions (hybrid mode)
10	Carney et al., 2020 [11]	Teachable Machine: no-code classifier trainer in the browser	Closest existing tool; this project adds a text mode and local ONNX inference
11	Jacob et al., 2018 [12]	8-bit integer quantization of neural networks	Basis for the quantized ONNX encoders

Table 2.1 Literature survey of few-shot, zero-shot and CLIP-based classification

2.4 RESEARCH GAP

The literature shows that strong pretrained embeddings make few-shot and zero-shot classification practical, but most published methods are evaluated in research code that depends on full deep-learning frameworks and GPUs, and the existing user-facing tools are limited to one input type or run in the cloud. A gap

therefore remains for a small, local, framework-free application that combines example-based and text-based classification over a single shared encoder and is evaluated honestly on real hardware. Teachable Lens is designed to fill that gap.

2.5 SUMMARY

This chapter reviewed vision transformers, CLIP, meta-learning, adaptation of CLIP, no-code teaching tools and quantization. The main lesson is that a good frozen embedding together with a simple trainable head is a strong, inexpensive recipe for few-shot classification, and that quantized ONNX models make it deployable on a CPU.

CHAPTER 3
EXISTING METHODS

3.1 INTRODUCTION

There are several established ways to obtain an image classifier for a custom set of categories. This chapter describes the main ones and explains why none of them fully meets the requirements of this project: user-defined classes, very few examples, CPU-only execution and local operation.

3.2 TRAINING A CONVOLUTIONAL NETWORK FROM SCRATCH

The classical approach is to design a convolutional neural network (CNN), collect a labelled dataset and train the network for many epochs with gradient descent. A CNN stacks convolution, pooling and fully connected layers to learn visual features automatically. The approach gives full control, but it needs thousands of labelled images per class, hours of training on a GPU and expertise in tuning. With only a few photos per class a network trained from scratch overfits badly.

3.3 FINE-TUNING A PRETRAINED NETWORK

Transfer learning with fine-tuning starts from a network pretrained on a large dataset and continues training all or part of its weights on the new data. It needs far fewer images than training from scratch, but it still requires a deep-learning framework, back-propagation through a large model and enough GPU memory to hold the weights, activations and optimiser state. On a 4 GB GPU, fine-tuning even a small vision transformer is constrained, and the resulting model forgets or must be retrained whenever a class is added.

3.4 META-LEARNING FOR FEW-SHOT CLASSIFICATION

Methods such as Matching Networks, Prototypical Networks and MAML [4]–[6] are designed for learning from few examples. They are trained on thousands of small episodes drawn from a large labelled dataset, which is

expensive and framework-dependent, and the resulting models are research artefacts rather than ready-to-use applications.

3.5 NO-CODE TEACHING TOOLS AND CLOUD SERVICES

Tools such as Teachable Machine [11] let users train a classifier from examples in the browser without writing code, and cloud machine-learning services allow users to upload a labelled dataset and receive a trained model. These tools are convenient, but they either rely on a fixed image-only feature extractor with no text-based option, or require uploading personal images to a remote service and depend on an internet connection.

3.6 ZERO-SHOT CLIP THROUGH FULL DEEP-LEARNING FRAMEWORKS

The reference implementation of CLIP runs on PyTorch. It supports zero-shot classification directly, but it requires installing PyTorch, which with CUDA takes several gigabytes, and it offers no interface for teaching personal classes from examples. Running the original floating-point weights is also heavier on a CPU than the 8-bit ONNX export used here.

3.7 LIMITATIONS OF EXISTING METHODS

Table 3.1 compares the approaches with the proposed system.

Approach	Labelled data needed	Hardware / software	Classes can change at run time
CNN from scratch	Thousands per class	GPU, deep-learning framework	No: retrain from the beginning
Fine-tuning a pretrained network	Tens to hundreds per class	GPU, deep-learning framework	No: retrain or fine-tune again
Meta-learning (MAML, Prototypical)	Few per class at test time; large dataset for pretraining	GPU, research code	Yes, within the trained episode format
No-code / cloud tools	A few to many per class	Browser or remote server	Yes, but images may leave the device
CLIP zero-shot (PyTorch)	None	PyTorch install of several GB	Yes, text only; no example-based mode
Teachable Lens (proposed)	**5–20 per class, or none**	**CPU, ONNX Runtime only, offline**	**Yes: example-based and text-based**

Table 3.1 Comparison of approaches for building a custom image classifier

3.8 SUMMARY

Existing methods either need a lot of labelled data and a GPU, depend on heavy frameworks, send images to the cloud, or support only one way of defining classes. The proposed system removes these obstacles by combining a quantized frozen encoder, a tiny trainable head and a text mode in one local application.

CHAPTER 4
PROPOSED SYSTEM

4.1 INTRODUCTION

Teachable Lens is a local application with a browser interface. Its core is a frozen CLIP ViT-B/32 encoder that converts images and text into 512-dimensional embeddings, and two small classification procedures that operate on those embeddings: a trainable logistic-regression head for few-shot classification and a cosine-similarity matcher for zero-shot classification. This chapter describes the architecture, the design decisions that follow from the hardware limits, the models used and the mathematics of the two modes.

4.2 SYSTEM ARCHITECTURE

Figure 4.1 shows the data flow. An input image from upload, webcam or clipboard is preprocessed and passed through the CLIP vision encoder to give an L2-normalised 512-dimensional embedding. From there the pipeline splits. In few-shot mode, the embeddings of the user's stored examples train the logistic-regression head, and the head classifies new embeddings. In zero-shot mode, the labels typed by the user are wrapped in a prompt, encoded by the CLIP text encoder, and compared with the image embedding by cosine similarity.

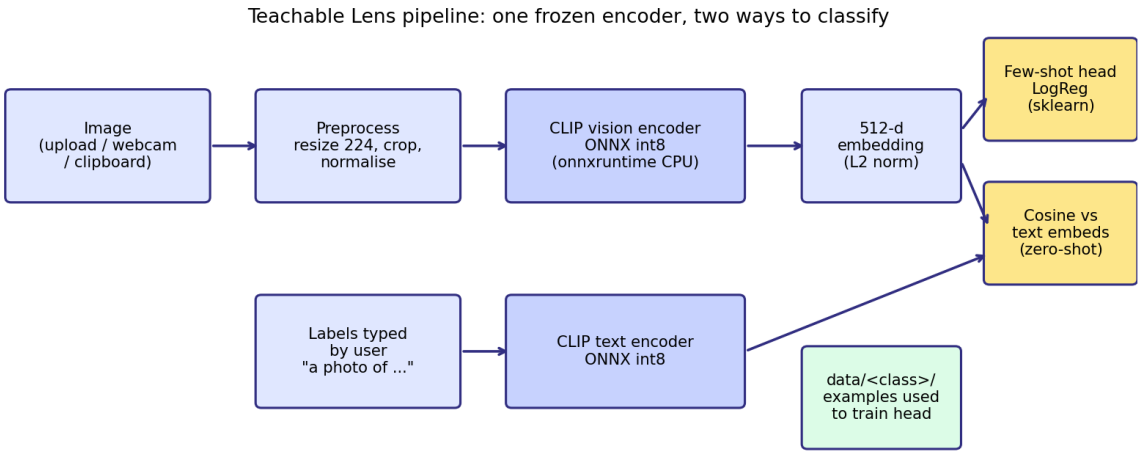


Figure 4.1 Proposed system architecture: both modes share the same frozen CLIP encoder

4.3 HARDWARE CONSTRAINTS AND DESIGN DECISIONS

The target machine has 4 GB of GPU memory and 16 GB of system memory. These constraints shaped every design choice, summarised in Table 4.1.

Decision	Choice	Reason
Inference engine	onnxruntime (CPU)	Small package; no CUDA or PyTorch needed
Backbone	CLIP ViT-B/32, int8 ONNX	Strong general features in a small quantized file
Training	Frozen encoder + logistic regression	Trains on the CPU in milliseconds; no GPU memory
Tokenizer	Hugging Face tokenizers (Rust)	Avoids installing the full transformers library
User interface	Gradio 6	Browser UI with webcam and clipboard input in pure Python
Storage	Plain folders + joblib	Dataset stays human-readable and portable

Table 4.1 Design decisions and their reasons

4.4 CLIP VIT-B/32 AS A FROZEN FEATURE EXTRACTOR

CLIP consists of two encoders trained together by a contrastive objective [2]. The vision encoder here is a Vision Transformer with 32×32 patches (ViT-B/32). It splits a 224×224 image into 7×7 = 49 patches, processes them with Transformer layers and projects the summary token to a 512-dimensional embedding. The text encoder is a Transformer that reads at most 77 tokens and also produces a 512-dimensional embedding.

Both encoders are used exactly as released: none of their weights is changed. Only the small classification head in few-shot mode is trained. This makes training independent of the size of the encoder and means that the heavy computation is done only once per image.

4.5 IMAGE PREPROCESSING

CLIP expects images in a specific format. The preprocessing used in this project reproduces it with Pillow and NumPy so that no deep-learning library is required (see Figure 4.2). The image is converted to RGB, its shorter side is

resized to 224 pixels using bicubic interpolation, the centre 224×224 region is cropped, the pixel values are scaled to the range 0–1, and each colour channel is normalised with CLIP's mean (0.4815, 0.4578, 0.4082) and standard deviation (0.2686, 0.2613, 0.2758). The result is a 3×224×224 array that is fed to the vision encoder.

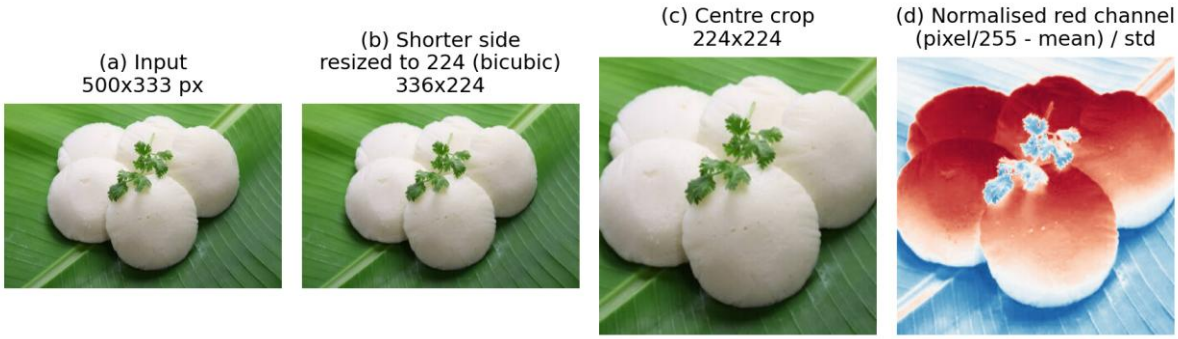


Figure 4.2 Image preprocessing steps for the CLIP vision encoder (real photograph from the project)

4.6 FEW-SHOT MODE: LOGISTIC-REGRESSION HEAD

In few-shot mode, the L2-normalised embeddings of all stored example images are used to train a multinomial logistic-regression classifier. For an embedding x and K classes, the probability of class k is given by the softmax function:

$$P(y = k \mid x) = \exp(w_k \cdot x + b_k) \,/\, \Sigma_j \exp(w_j \cdot x + b_j)$$

The weights w_k and biases b_k are found by minimising the cross-entropy loss with an L2 penalty on the weights. The implementation uses scikit-learn's LogisticRegression [14] with inverse regularisation strength C = 10 (a weak penalty, appropriate because normalised embeddings have small values), up to 2000 iterations of the default L-BFGS solver, and class-balanced weights so that a class with fewer photos is not ignored.

A linear classifier on a good embedding is attractive here for four reasons. It is convex, so training always converges to the same solution. It is fast, so

retraining after every change takes milliseconds. It produces calibrated-looking probabilities that can be shown as confidence bars. And, because the encoder is frozen, adding a class means adding a folder and retraining, with no risk of forgetting the old classes. The trained head is saved to models/head.joblib and reloaded when the application starts.

4.7 ZERO-SHOT MODE: TEXT PROMPTS

In zero-shot mode the user types comma-separated labels. Each label is wrapped in the prompt template “a photo of {label}” and encoded by the CLIP text encoder, giving text embeddings t_1 … t_K. The score of each label is the cosine similarity between the image embedding v and the text embedding, multiplied by CLIP’s learned logit scale of 100, and the scores are converted to probabilities with a softmax:

$$s_k = 100 \cdot (t_k \cdot v), \quad P(label = k) = exp(s_k) / \Sigma_j exp(s_j)$$

Because all embeddings are L2-normalised, the dot product equals the cosine similarity. No training is involved: the classifier changes as soon as the user edits the label text.

4.8 MODELS USED (HUGGING FACE)

All pretrained weights come from the Hugging Face Hub and are downloaded once into the local cache. No model is trained from scratch; only the small classification head is trained locally. Table 4.2 lists the files (model cards: [16]).

Component	Hugging Face repository / file	Size
Vision encoder	Xenova/clip-vit-base-patch32 onnx/vision_model_quantized.onnx	85 MB
Text encoder	Xenova/clip-vit-base-patch32 onnx/text_model_quantized.onnx	62 MB
Tokenizer	Xenova/clip-vit-base-patch32 tokenizer.json	< 5 MB
Original weights	openai/clip-vit-base-patch32 (MIT licence)	–

Table 4.2 Pretrained models and files used from the Hugging Face Hub

The Xenova repository is a community ONNX export of the official OpenAI checkpoint with 8-bit weight quantization [12]. Quantization reduces the file size by roughly a factor of four and speeds up CPU inference at the cost of a small drop in accuracy.

4.9 SUMMARY

The proposed system uses a frozen, quantized CLIP encoder to turn images and text into embeddings. A logistic-regression head trained on a few example embeddings gives personalised classification, and cosine similarity between image and text embeddings gives training-free classification. The design keeps every component within the limits of a 4 GB GPU / 16 GB RAM laptop.

CHAPTER 5
IMPLEMENTATION AND TOOLS

This chapter describes the software environment, the tools used and the structure of the code. The complete source code is reproduced in the Appendix.

5.1 SOFTWARE ENVIRONMENT

The application is written in Python 3.12. The exact versions of the libraries are pinned in requirements.txt and listed in Table 5.1. Neither PyTorch nor TensorFlow is required.

Library	Version	Purpose
onnxruntime	1.30.0	Runs the quantized CLIP encoders on the CPU
numpy	1.26.4	Array handling, normalisation, softmax
pillow	12.2.0	Image loading, resizing and cropping
tokenizers	0.22.2	CLIP text tokenizer (Rust implementation)
huggingface_hub	1.33.0	Downloads and caches the ONNX files
scikit-learn	1.7.1	Logistic-regression head
joblib	1.5.1	Saves and loads the trained head
gradio	6.29.0	Browser user interface

Table 5.1 Libraries used and their versions

5.2 ONNX RUNTIME

ONNX (Open Neural Network Exchange) is an open file format for machine-learning models, and ONNX Runtime is a high-performance engine that executes them [13]. The application creates one inference session per encoder with the full graph-optimisation level and the CPU execution provider. Because the models are stored in the framework-neutral ONNX format, the application needs neither PyTorch nor TensorFlow.

5.3 HUGGING FACE HUB AND TOKENIZERS

The huggingface_hub library downloads the ONNX files and the tokenizer definition on first use and caches them in the user's home directory, so later runs need no network access. The tokenizers library, which is implemented in Rust,

loads CLIP's byte-pair-encoding vocabulary from tokenizer.json and turns a prompt into the integer token IDs the text encoder expects. Using it alone avoids installing the much larger transformers package.

5.4 SCIKIT-LEARN AND JOBLIB

scikit-learn [14] supplies the LogisticRegression class used as the few-shot head. joblib serialises the fitted classifier together with the list of class names, which makes loading the model at start-up a single function call.

5.5 GRADIO

Gradio [15] builds the web interface in pure Python. It provides image components with upload, webcam and clipboard sources, a file component for multiple uploads, a data table, buttons, and a label component that renders the top classes as confidence bars. Errors raised with gr.Error appear as toast messages.

5.6 MODULE STRUCTURE

The code is split into three short modules of about 250 lines in total (Table 5.2).

File	Responsibility
lens/clip_onnx.py	Downloads the ONNX files; image preprocessing; image and text embedding
lens/head.py	FewShotHead class: fit, predict, save and load the logistic-regression head
app.py	Gradio interface, dataset-folder management, training and classification callbacks

Table 5.2 Source files and their responsibilities

5.7 EMBEDDING AN IMAGE WITHOUT PYTORCH

CLIP's preprocessing is reimplemented with Pillow and NumPy, so the only run-time dependency for the model is onnxruntime. The core of lens/clip_onnx.py is shown below.

```
def preprocess(img):
    img = img.convert("RGB")
    w, h = img.size
    scale = SIZE / min(w, h)                        # shorter side -> 224
```

```
    img = img.resize((max(SIZE, round(w*scale)), max(SIZE, round(h*scale))),
Image.BICUBIC)
    ...                                             # centre crop to 224 x 224
    arr = (np.asarray(img, dtype=np.float32) / 255.0 - MEAN) / STD
    return arr.transpose(2, 0, 1)                   # HWC -> CHW

def embed_images(self, images, batch=16):
    for i in range(0, len(images), batch):
        pixels = np.stack([self.preprocess(im) for im in images[i:i+batch]])
        out.append(self.vision.run([self._vision_out], {self._vision_in:
pixels})[0])
    return _normalize(np.concatenate(out).astype(np.float32))   # L2-normalised
```

5.8 TRAINING THE HEAD

Training embeds every image under data/<class>/ in batches of 16 and fits the logistic regression. Because the encoder is frozen, the heavy computation happens exactly once per image; fitting the head itself takes only a few milliseconds. The status line after training reports the number of images, the number of classes and the training accuracy. Training is refused with a clear message if fewer than two classes exist.

```
self.clf = LogisticRegression(C=10.0, max_iter=2000, class_weight="balanced")
self.clf.fit(embeddings, labels)
self.classes = list(self.clf.classes_)
```

5.9 DATASET STORAGE

Example images are copied into one folder per class under data/, with sequential file names. The dataset can therefore be inspected, backed up or edited in a normal file manager, and the Teach tab shows a live table with the number of images in each class. Deleting a class removes its folder, after which the model is retrained.

5.10 USER INTERFACE

The Gradio interface has three tabs. **Teach** manages the classes: the user types a class name, uploads images, presses Add examples and finally Train model. **Classify** runs the trained model on an uploaded, webcam or pasted image and shows the top-five classes as confidence bars that update as soon as the image changes. **Zero-shot** accepts a comma-separated list of labels and needs no training. Screenshots are shown in Section 6.6.

5.11 SUMMARY

The application is a small Python program built from standard open-source libraries. ONNX Runtime executes the quantized encoders, scikit-learn supplies the trainable head, and Gradio provides the interface. The code base is about 250 lines, and no deep-learning framework is needed at run time.

CHAPTER 6
RESULTS AND DISCUSSIONS

6.1 EVALUATION METHODOLOGY

Three kinds of measurement were made: (i) accuracy on a synthetic shape benchmark with a held-out test set, (ii) accuracy on real photographs using leave-one-out cross-validation and repeated random splits, and (iii) speed and memory use on the target hardware. All timings use onnxruntime's CPU execution provider after one warm-up call.

The main metric is **accuracy**, the fraction of test images that are classified correctly:

Accuracy = (number of correct predictions) / (total number of predictions)

For a class c, **precision** is TP / (TP + FP) and **recall** is TP / (TP + FN), where TP, FP and FN are the true positives, false positives and false negatives; the **F1 score** is their harmonic mean, 2 · Precision · Recall / (Precision + Recall). Because most results below are perfect or near-perfect, confusion matrices and per-image results are shown in addition to accuracy.

6.2 EXPERIMENT 1: SYNTHETIC SHAPE BENCHMARK

To obtain reproducible numbers without shipping a third-party dataset, a synthetic benchmark was generated: three shape classes (circle, square, triangle) drawn with a random colour out of ten, a random size, a random position and one of four background tints (Figure 6.1). Colour is deliberately uninformative, so the classifier has to learn shape. The test set holds 90 images (30 per class) that never appear in training.

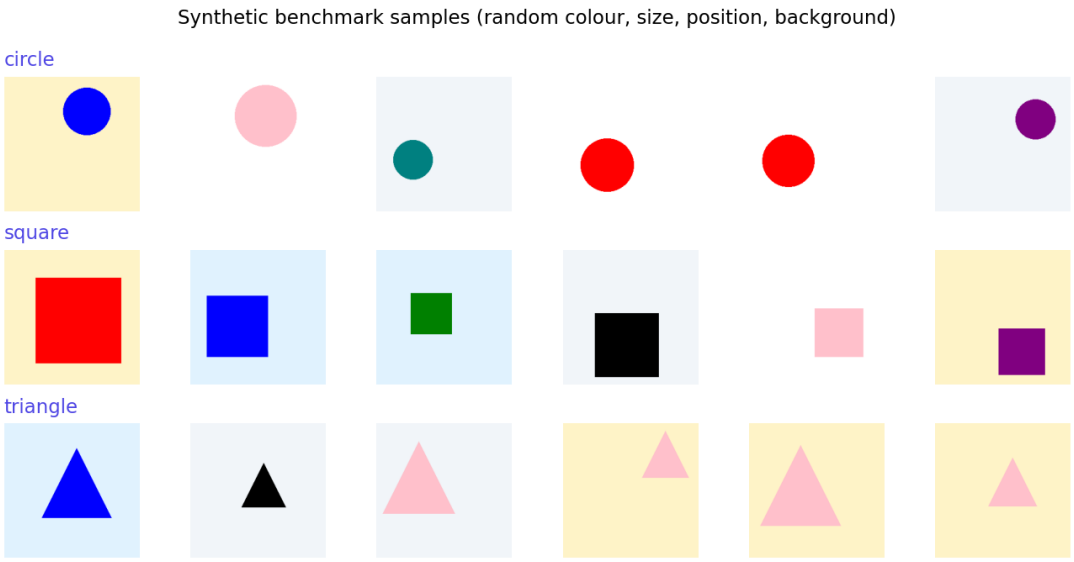


Figure 6.1 Example images from the synthetic benchmark

For each budget of k images per class, five random training subsets were drawn from a pool of 20 images per class. Table 6.1 and Figure 6.2 report the mean and standard deviation of the test accuracy.

Images per class	1	2	5	10	20
Accuracy (mean)	87.1%	96.2%	98.7%	99.8%	100.0%
Standard deviation	±5.9%	±2.5%	±2.7%	±0.4%	±0.0%

Table 6.1 Few-shot accuracy on the synthetic benchmark (90 test images, 5 random subsets)

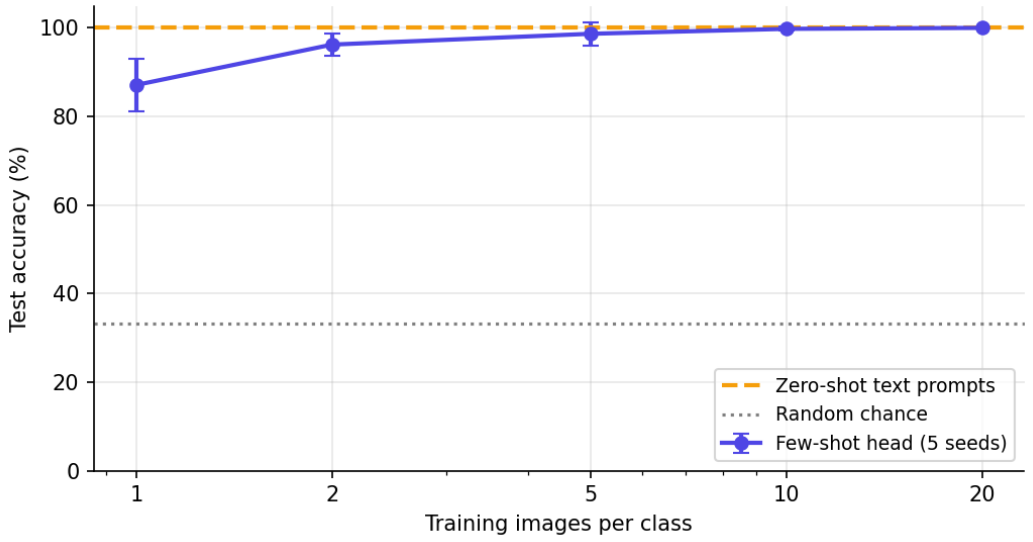


Figure 6.2 Few-shot accuracy versus number of examples per class, with the zero-shot baseline

With a single example per class the head already reaches 87.1%, well above the 33% chance level for three classes. Accuracy rises quickly and reaches 99.8% with 10 examples and 100% with 20. The shrinking standard deviation shows that the result becomes less dependent on which examples were chosen as more are added.

With the prompts “a photo of a circle / square / triangle” and no training, the zero-shot mode also reached 100.0%, and the few-shot head with 10 examples per class reached 100.0% (Figure 6.3). Simple shapes have short, precise names that CLIP learned from its training captions, so the text prompts alone are enough.

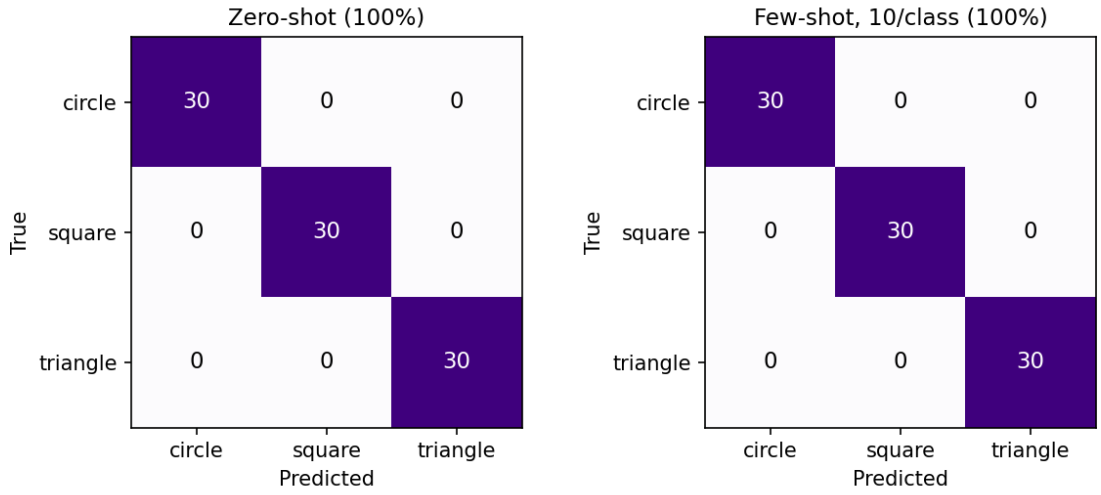


Figure 6.3 Confusion matrices on the 90-image test set (zero-shot and few-shot with 10 examples per class)

6.3 EXPERIMENT 2: REAL PHOTOGRAPHS (IDLY VERSUS RICE)

To check that the pipeline also works on real photographs, the demonstration dataset of the application was evaluated. It contains 20 web photographs of two South-Indian foods: 10 of idly (steamed rice cakes, often served on banana leaf) and 10 of plain cooked rice in bowls and on plates. These are the same 20 images that the application has in its data/ folder and in test-images/. Training the model in the application on all 20 images reports a training accuracy of 100% (Section 6.6), but **training accuracy is not a measure of generalisation**, because the model has seen those very images.

A held-out estimate was therefore obtained in two ways on the same 20 images. In **leave-one-out cross-validation**, each image in turn is removed, the head is trained on the other 19 and the removed image is classified. In **repeated random splits**, k images per class are drawn for training and the remaining images are used for testing, repeated 50 times for each k.

Leave-one-out accuracy was **100%** (20 of 20). The mean confidence of the correct predictions was 81%, with values between 57% and 91% (Figure 6.4). The accuracy for smaller training sets is given in Table 6.2 and Figure 6.5.



Figure 6.4 Leave-one-out predictions on the 20 real photographs (green = correct); #0–9 are rice, #10–19 are idly

Training images per class	1	2	3	5	8
Held-out accuracy (mean)	91.9%	97.3%	98.1%	99.4%	100.0%
Standard deviation	±7.2%	±3.1%	±3.1%	±2.4%	±0.0%

Table 6.2 Few-shot accuracy on real photographs (idly vs rice), 50 random splits per setting

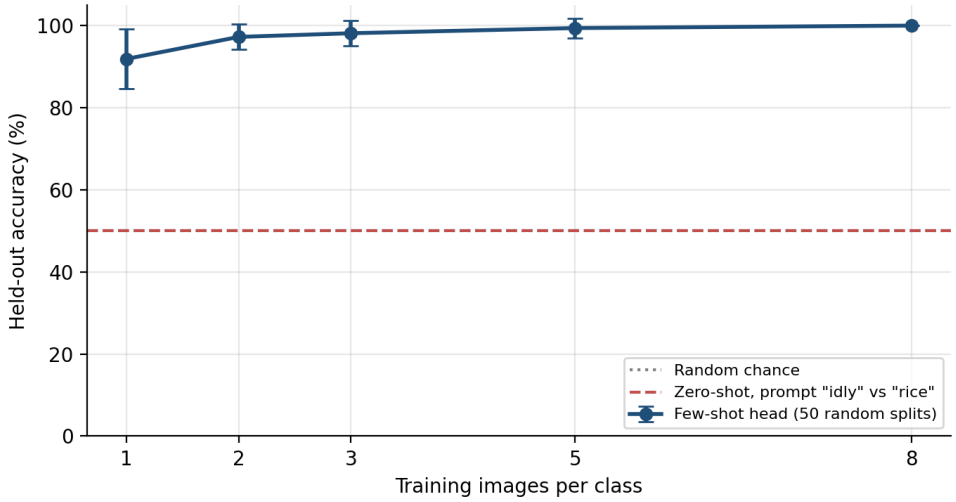


Figure 6.5 Held-out accuracy versus number of training images per class on real photographs

The trend matches the synthetic benchmark: one photo per class already gives about 92% and five give 99%. The result must be read with care (see Section 6.7): idly and rice look very different, the sample is small, and part of the separation may come from backgrounds (banana leaf versus bowls and plates).

6.4 ZERO-SHOT VERSUS FEW-SHOT CLASSIFICATION

The zero-shot mode was tested on the same 20 photographs with the label pairs shown in Table 6.3. Zero-shot needs no training, so every image is a fair test image. The results expose the main weakness of text-based classification: it depends on the wording of the prompts.

Labels typed by the user	Accuracy on 20 photos	Observation
idly, rice	50% (10/20)	All 20 photos were labelled “rice”
idli, rice	50% (10/20)	The common alternative spelling gives the same accuracy
steamed idli, cooked white rice	50% (10/20)	More words alone did not help
idli (steamed rice cake), plain cooked rice	100% (20/20)	A descriptive label gave a correct result

Table 6.3 Zero-shot accuracy on the 20 real photographs for different label wording

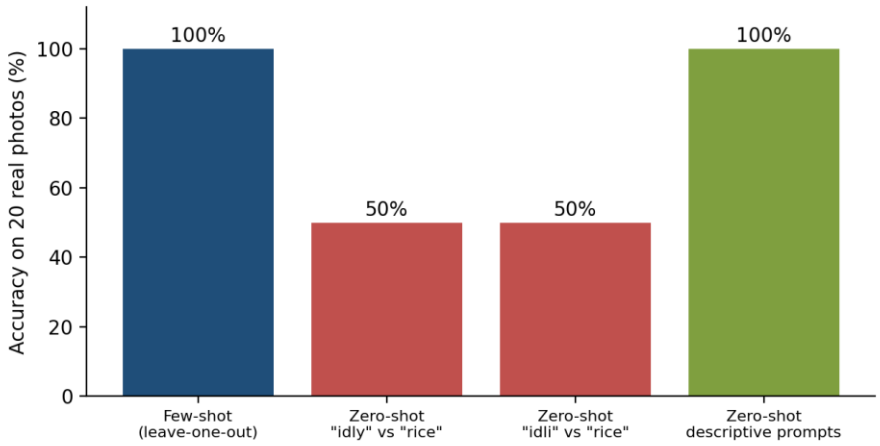


Figure 6.6 Few-shot (leave-one-out) versus zero-shot accuracy on 20 real photographs

The word “idly” (or “idli”) is rare in the captions CLIP was trained on, so the text embedding of “a photo of idly” is not close to the images of idly, and CLIP chose “rice” for every photo, which gives exactly 50%. Describing the food in common words (“steamed rice cake”) fixed the problem. The descriptive wording, however, was chosen after seeing that the simple names failed, and the sample is only 20 images, so that 100% should be treated as an illustration of prompt sensitivity and not as an unbiased accuracy estimate.

This is exactly the situation in which the few-shot mode is valuable: ten photos per class were enough to reach 100% leave-one-out accuracy without any prompt engineering. Zero-shot works when a category has a clear everyday name and needs nothing but words. Few-shot makes the application personal: categories such as “my bike” versus “my friend’s bike”, or a specific logo, cannot

be described in text but can be learned from a few photos. The two modes complement each other in the same application.

6.5 PERFORMANCE ON THE TARGET HARDWARE

All timings were measured on the development laptop using onnxruntime's CPU execution provider after one warm-up call (Table 6.4, Figure 6.7).

Metric	Value
Image embedding, single image	23 ms
Image embedding, batch of 16	21 ms per image
Text embedding (one prompt)	4 ms
Head training, 30 images	8 ms (after embedding)
Process memory (RSS) after the benchmark	475 MB
GPU memory used	0 MB (CPU only)
Model download (one time)	147 MB

Table 6.4 Measured performance on the target laptop

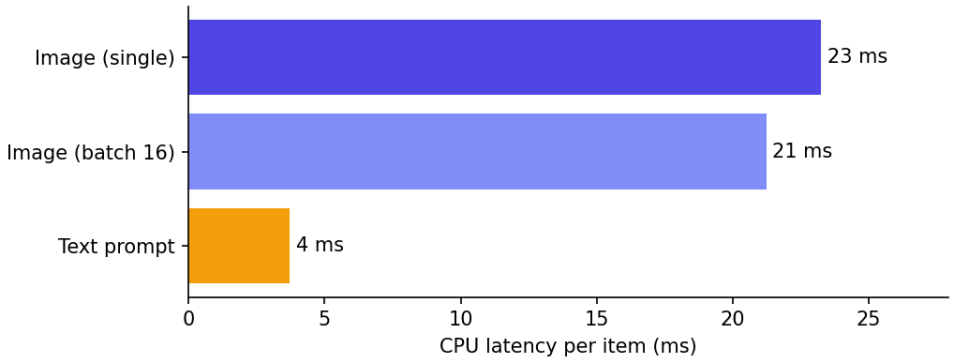


Figure 6.7 Average CPU latency per item

A later run on the 20 idly/rice photographs gave 18 ms per image for embedding and 6 ms for fitting the head on 20 images, consistent with the table. The whole application fits comfortably within the 16 GB RAM budget and leaves the 4 GB GPU free. Latency is low enough for the Classify tab to update interactively when an image is uploaded.

6.6 APPLICATION OUTPUTS

Figure 6.8 shows the Teach tab after the two classes were added and the model was trained: the table lists the classes and image counts, and the status line reports “Trained on 20 images, 2 classes (train accuracy 100%)”.

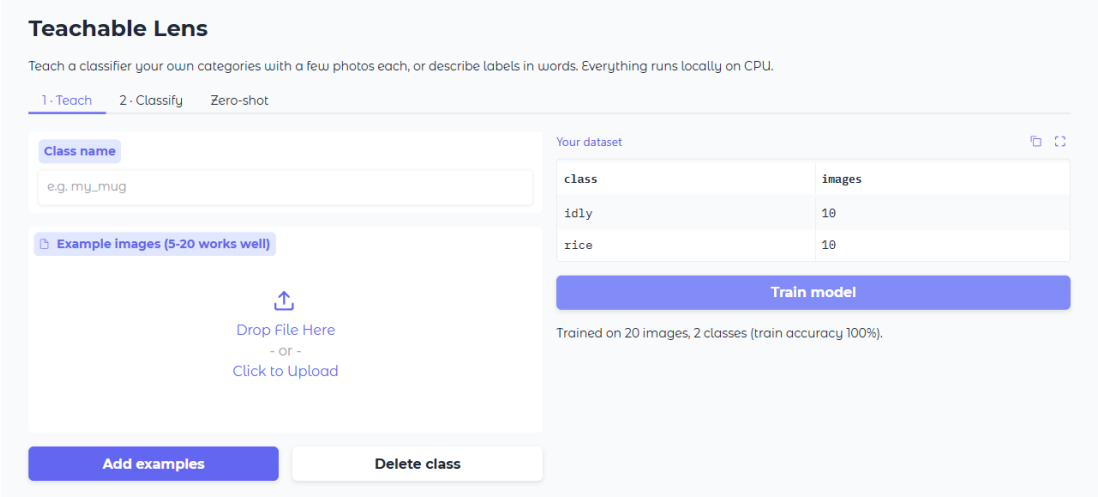


Figure 6.8 Teach tab: dataset table and training status

In the Classify tab (Figure 6.9), an uploaded photograph of idly is classified immediately by the trained model as “idly” with 89% confidence, with the remaining 11% going to “rice”.

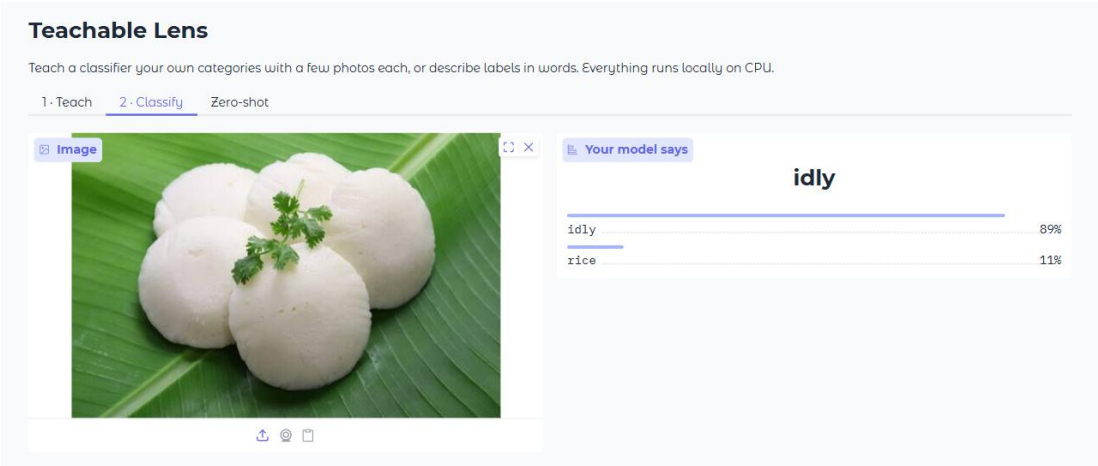


Figure 6.9 Classify tab: output for an idly photograph

In the Zero-shot tab (Figure 6.10), a photograph of cooked rice is matched against the typed labels “rice, idly, a cat, a car” and is assigned to “rice” with essentially 100% probability.

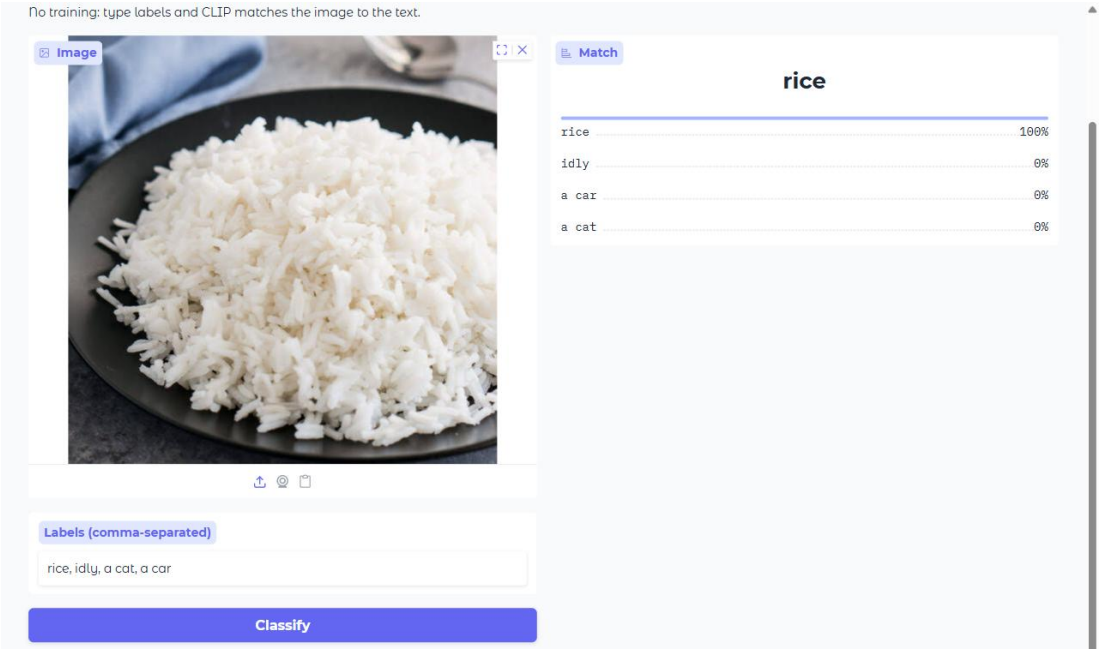


Figure 6.10 Zero-shot tab: output for a photograph of cooked rice

6.7 LIMITATIONS OF THE EVALUATION

- The synthetic shapes are much simpler than real photographs, so Experiment 1 shows that the pipeline works and how accuracy scales with the number of examples, but it does not predict accuracy on arbitrary user data.
- Experiment 2 uses only 20 images of two visually very different foods, and backgrounds may help the classifier. The numbers are encouraging but are not a general benchmark.
- The test-images/ folder duplicates the training images in data/, so it cannot serve as an independent test set; leave-one-out cross-validation was used instead.
- Zero-shot results depend on the wording of the labels, and the best wording was found after seeing the results.
- Evaluating on a public few-shot benchmark with many classes and fine-grained categories is a natural next step.

6.8 SUMMARY

On the synthetic benchmark, the few-shot head reaches 87% accuracy with one example per class and 100% with twenty. On real photographs, leave-one-out accuracy is 100% for idly versus rice, and the zero-shot mode shows how strongly text-based classification depends on label wording. An image is embedded in about 23 ms on the CPU, the head trains in milliseconds and the application uses under 500 MB of memory.

CHAPTER 7
CONCLUSION AND FUTURE WORK

Teachable Lens shows that a useful, personal image classifier does not need a GPU framework or a large labelled dataset. A quantized CLIP encoder running in ONNX Runtime supplies general-purpose features, and a logistic-regression head trained in milliseconds turns them into a classifier for any user-defined classes. A second, training-free mode classifies images against text labels using the same encoder. On the synthetic benchmark the system reached 99% accuracy with five examples per class and 100% with twenty, and on 20 real photographs it reached 100% leave-one-out accuracy, all within the CPU and memory budget of a laptop with 4 GB of GPU memory and 16 GB of RAM. The experiments also showed the limits of the zero-shot mode, whose accuracy depends on how the labels are worded, and confirmed that the two modes complement each other.

The following improvements are possible in future work:

- **Fine-grained classes.** Frozen CLIP features can struggle with very similar classes, such as two nearly identical products. Fine-tuning the encoder would help but needs a GPU framework.
- **Quantization.** The int8 weights trade some accuracy for speed. An fp32 ONNX file can be swapped in by changing one constant.
- **GPU acceleration.** Installing onnxruntime-directml would let Windows use the 4 GB GPU without PyTorch.
- **Hybrid mode.** Text embeddings of the class names could be added as extra training points, so that classes work well with only one or two photos, in the spirit of Tip-Adapter [10].
- **Real benchmark.** Evaluating on a public few-shot dataset would give a more general accuracy estimate.

- **Export.** The trained head could be exported to ONNX (for example with skl2onnx) for use on mobile devices or in the web browser.

REFERENCES

[1] Y. LeCun, Y. Bengio and G. Hinton, “Deep learning,” Nature, vol. 521, pp. 436–444, 2015.

[2] A. Radford, J. W. Kim, C. Hallacy et al., “Learning transferable visual models from natural language supervision,” in Proc. 38th Int. Conf. Machine Learning (ICML), PMLR 139, 2021, pp. 8748–8763.

[3] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in Proc. Int. Conf. Learning Representations (ICLR), 2021.

[4] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu and D. Wierstra, “Matching networks for one shot learning,” in Advances in Neural Information Processing Systems (NeurIPS), 2016.

[5] J. Snell, K. Swersky and R. Zemel, “Prototypical networks for few-shot learning,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[6] C. Finn, P. Abbeel and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in Proc. 34th Int. Conf. Machine Learning (ICML), 2017.

[7] Y. Tian, Y. Wang, D. Krishnan, J. B. Tenenbaum and P. Isola, “Rethinking few-shot image classification: a good embedding is all you need?,” in Proc. European Conf. Computer Vision (ECCV), 2020.

[8] K. Zhou, J. Yang, C. C. Loy and Z. Liu, “Learning to prompt for vision-language models,” Int. J. Computer Vision, vol. 130, pp. 2337–2348, 2022.

[9] P. Gao et al., “CLIP-Adapter: better vision-language models with feature adapters,” arXiv:2110.04544, 2021.

[10] R. Zhang et al., “Tip-Adapter: training-free adaption of CLIP for few-shot classification,” in Proc. European Conf. Computer Vision (ECCV), 2022.

[11] M. Carney et al., “Teachable Machine: approachable web-based tool for exploring machine learning classification,” in Extended Abstracts of the CHI Conf. Human Factors in Computing Systems, 2020.

[12] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018.

[13] ONNX Runtime developers, “ONNX Runtime,” https://onnxruntime.ai.

[14] F. Pedregosa et al., “Scikit-learn: machine learning in Python,” J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[15] A. Abid et al., “Gradio: hassle-free sharing and testing of ML models in the wild,” arXiv:1906.02569, 2019.

[16] OpenAI, “openai/clip-vit-base-patch32,” and Xenova, “Xenova/clip-vit-base-patch32 (ONNX export),” Hugging Face Hub, https://huggingface.co.

APPENDIX

A. SOURCE CODE

The complete source code of the application is listed below. It is also available in the project folder.

A.1 lens/clip_onnx.py

```
"""CLIP ViT-B/32 running through onnxruntime (no PyTorch).

Model: Xenova/clip-vit-base-patch32 on Hugging Face (quantized ONNX export of
openai/clip-vit-base-patch32). Files are downloaded once into the HF cache.
"""
from __future__ import annotations

import numpy as np
import onnxruntime as ort
from huggingface_hub import hf_hub_download
from PIL import Image
from tokenizers import Tokenizer

REPO_ID = "Xenova/clip-vit-base-patch32"
VISION_FILE = "onnx/vision_model_quantized.onnx"
TEXT_FILE = "onnx/text_model_quantized.onnx"

# CLIP preprocessing constants
SIZE = 224
MEAN = np.array([0.48145466, 0.4578275, 0.40821073], dtype=np.float32)
STD = np.array([0.26862954, 0.26130258, 0.27577711], dtype=np.float32)


def _session(path: str) -> ort.InferenceSession:
    opts = ort.SessionOptions()
    opts.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
    return ort.InferenceSession(path, opts, providers=["CPUExecutionProvider"])


def _embeds_output(sess: ort.InferenceSession) -> str:
    names = [o.name for o in sess.get_outputs()]
    return next((n for n in names if n.endswith("embeds")), names[0])


def _normalize(x: np.ndarray) -> np.ndarray:
    return x / np.linalg.norm(x, axis=-1, keepdims=True)


class ClipOnnx:
    def __init__(self) -> None:
        self.vision = _session(hf_hub_download(REPO_ID, VISION_FILE))
        self.text = _session(hf_hub_download(REPO_ID, TEXT_FILE))
        self.tokenizer = Tokenizer.from_file(hf_hub_download(REPO_ID, "tokenizer.json"))
        self._vision_in = self.vision.get_inputs()[0].name
        self._vision_out = _embeds_output(self.vision)
        self._text_in = self.text.get_inputs()[0].name
        self._text_out = _embeds_output(self.text)

    @staticmethod
    def preprocess(img: Image.Image) -> np.ndarray:
        img = img.convert("RGB")
        w, h = img.size
        scale = SIZE / min(w, h)
        img = img.resize((max(SIZE, round(w * scale)), max(SIZE, round(h * scale))),
Image.BICUBIC)
        w, h = img.size
        left, top = (w - SIZE) // 2, (h - SIZE) // 2
        img = img.crop((left, top, left + SIZE, top + SIZE))
        arr = (np.asarray(img, dtype=np.float32) / 255.0 - MEAN) / STD
```

```
        return arr.transpose(2, 0, 1)  # HWC -> CHW

    def embed_images(self, images: list[Image.Image], batch: int = 16) -> np.ndarray:
        out = []
        for i in range(0, len(images), batch):
            pixels = np.stack([self.preprocess(im) for im in images[i : i + batch]])
            out.append(self.vision.run([self._vision_out], {self._vision_in: pixels})[0])
        return _normalize(np.concatenate(out).astype(np.float32))

    def embed_texts(self, texts: list[str]) -> np.ndarray:
        # One prompt at a time avoids padding/attention-mask differences between exports.
        out = []
        for t in texts:
            ids = self.tokenizer.encode(t).ids[:77]
            ids = np.array([ids], dtype=np.int64)
            out.append(self.text.run([self._text_out], {self._text_in: ids})[0])
        return _normalize(np.concatenate(out).astype(np.float32))
```

A.2 lens/head.py

```
"""Tiny trainable classifier on top of frozen CLIP embeddings."""
from __future__ import annotations

from pathlib import Path

import joblib
import numpy as np
from sklearn.linear_model import LogisticRegression

HEAD_PATH = Path(__file__).resolve().parent.parent / "models" / "head.joblib"


class FewShotHead:
    def __init__(self) -> None:
        self.clf: LogisticRegression | None = None
        self.classes: list[str] = []

    @property
    def ready(self) -> bool:
        return self.clf is not None

    def fit(self, embeddings: np.ndarray, labels: list[str]) -> None:
        # Strong C: embeddings are L2-normalized, so features are small.
        self.clf = LogisticRegression(C=10.0, max_iter=2000, class_weight="balanced")
        self.clf.fit(embeddings, labels)
        self.classes = list(self.clf.classes_)

    def predict(self, embedding: np.ndarray) -> dict[str, float]:
        probs = self.clf.predict_proba(embedding.reshape(1, -1))[0]
        return {c: float(p) for c, p in zip(self.classes, probs)}

    def save(self, path: Path = HEAD_PATH) -> None:
        path.parent.mkdir(parents=True, exist_ok=True)
        joblib.dump({"clf": self.clf, "classes": self.classes}, path)

    def load(self, path: Path = HEAD_PATH) -> bool:
        if not path.exists():
            return False
        data = joblib.load(path)
        self.clf, self.classes = data["clf"], data["classes"]
        return True
```

A.3 app.py

```
"""Teachable Lens: teach your own image classes with a few photos, or classify zero-shot.

Runs fully local on CPU: CLIP (ONNX, from Hugging Face) + a tiny scikit-learn head.
"""
from __future__ import annotations

import re
import shutil
from pathlib import Path
```

```
import gradio as gr
from PIL import Image

from lens.clip_onnx import ClipOnnx
from lens.head import FewShotHead

DATA_DIR = Path(__file__).resolve().parent / "data"
IMG_EXT = {".jpg", ".jpeg", ".png", ".webp", ".bmp"}

_clip: ClipOnnx | None = None
head = FewShotHead()
head.load()


def clip() -> ClipOnnx:
    global _clip
    if _clip is None:
        _clip = ClipOnnx()  # first call downloads ~150 MB of ONNX weights
    return _clip


def _slug(name: str) -> str:
    return re.sub(r"[^a-z0-9_-]+", "_", name.strip().lower()).strip("_")


def dataset_summary() -> list[list]:
    if not DATA_DIR.exists():
        return []
    rows = []
    for d in sorted(p for p in DATA_DIR.iterdir() if p.is_dir()):
        n = sum(1 for f in d.iterdir() if f.suffix.lower() in IMG_EXT)
        rows.append([d.name, n])
    return rows


def add_examples(class_name: str, files: list[str] | None):
    slug = _slug(class_name or "")
    if not slug:
        raise gr.Error("Give the class a name first.")
    if not files:
        raise gr.Error("Upload at least one image.")
    target = DATA_DIR / slug
    target.mkdir(parents=True, exist_ok=True)
    start = len(list(target.iterdir()))
    added = 0
    for i, f in enumerate(files):
        src = Path(f)
        if src.suffix.lower() in IMG_EXT:
            shutil.copy(src, target / f"{start + i:04d}{src.suffix.lower()}")
            added += 1
    return dataset_summary(), None, f"Added {added} image(s) to '{slug}'."


def delete_class(class_name: str):
    target = DATA_DIR / _slug(class_name or "")
    if not class_name or not target.is_dir():
        raise gr.Error("No class with that name.")
    shutil.rmtree(target)
    return dataset_summary(), f"Deleted '{target.name}'. Retrain to update the model."


def train():
    images, labels = [], []
    for name, _ in dataset_summary():
        for f in sorted((DATA_DIR / name).iterdir()):
            if f.suffix.lower() in IMG_EXT:
                images.append(Image.open(f))
                labels.append(name)
    if len(set(labels)) < 2:
        raise gr.Error("You need at least 2 classes to train.")
    emb = clip().embed_images(images)
    head.fit(emb, labels)
    head.save()
    acc = head.clf.score(emb, labels)
```

```
    return f"Trained on {len(labels)} images, {len(set(labels))} classes (train accuracy
{acc:.0%})."


def classify_custom(img: Image.Image | None):
    if img is None:
        return None
    if not head.ready:
        raise gr.Error("Train a model in the Teach tab first.")
    return head.predict(clip().embed_images([img])[0])


def classify_zero_shot(img: Image.Image | None, labels_text: str):
    if img is None:
        return None
    labels = [l.strip() for l in (labels_text or "").split(",") if l.strip()]
    if len(labels) < 2:
        raise gr.Error("Type at least 2 comma-separated labels.")
    img_emb = clip().embed_images([img])[0]
    txt_emb = clip().embed_texts([f"a photo of {l}" for l in labels])
    logits = 100.0 * txt_emb @ img_emb  # CLIP's learned logit scale
    probs = np_softmax(logits)
    return {l: float(p) for l, p in zip(labels, probs)}


def np_softmax(x):
    import numpy as np

    e = np.exp(x - x.max())
    return e / e.sum()


with gr.Blocks(title="Teachable Lens") as demo:
    gr.Markdown(
        "# Teachable Lens\n"
        "Teach a classifier your own categories with a few photos each, or describe labels
in words. "
        "Everything runs locally on CPU."
    )
    with gr.Tab("1 · Teach"):
        with gr.Row():
            with gr.Column():
                cls_name = gr.Textbox(label="Class name", placeholder="e.g. my_mug")
                uploads = gr.File(label="Example images (5-20 works well)",
file_count="multiple",
                                  file_types=["image"], type="filepath")
                with gr.Row():
                    add_btn = gr.Button("Add examples", variant="primary")
                    del_btn = gr.Button("Delete class", variant="stop")
            with gr.Column():
                table = gr.Dataframe(headers=["class", "images"], value=dataset_summary(),
                                     interactive=False, label="Your dataset")
                train_btn = gr.Button("Train model", variant="primary")
                status = gr.Markdown()
        add_btn.click(add_examples, [cls_name, uploads], [table, uploads, status])
        del_btn.click(delete_class, [cls_name], [table, status])
        train_btn.click(train, None, status)

    with gr.Tab("2 · Classify"):
        with gr.Row():
            img_in = gr.Image(type="pil", label="Image", sources=["upload", "webcam",
"clipboard"])
            out = gr.Label(num_top_classes=5, label="Your model says")
        img_in.change(classify_custom, img_in, out)

    with gr.Tab("Zero-shot"):
        gr.Markdown("No training: type labels and CLIP matches the image to the text.")
        with gr.Row():
            with gr.Column():
                zs_img = gr.Image(type="pil", label="Image", sources=["upload", "webcam",
"clipboard"])
                zs_labels = gr.Textbox(label="Labels (comma-separated)", value="a cat, a
dog, a car, food")
                zs_btn = gr.Button("Classify", variant="primary")
            zs_out = gr.Label(num_top_classes=5, label="Match")
```

```
        zs_btn.click(classify_zero_shot, [zs_img, zs_labels], zs_out)

if __name__ == "__main__":
    demo.launch(theme=gr.themes.Soft())
```

A.4 requirements.txt

```
gradio==6.29.0
huggingface_hub==1.33.0
joblib==1.5.1
numpy==1.26.4
onnxruntime==1.30.0
pillow==12.2.0
scikit-learn==1.7.1
tokenizers==0.22.2
```

B. PER-IMAGE RESULTS

Table A.1 lists, for each of the 20 real photographs, the leave-one-out prediction of the few-shot head (trained on the other 19 images) and the zero-shot prediction with the labels “idly” and “rice”. Images #0–9 are rice and #10–19 are idly.

Image	True class	Few-shot (leave-one-out)	Confidence	Zero-shot P(idly)	Zero-shot result
#0	rice	rice ✓	80%	0.0%	rice ✓
#1	rice	rice ✓	74%	0.1%	rice ✓
#2	rice	rice ✓	83%	0.0%	rice ✓
#3	rice	rice ✓	88%	0.0%	rice ✓
#4	rice	rice ✓	88%	0.0%	rice ✓
#5	rice	rice ✓	86%	0.0%	rice ✓
#6	rice	rice ✓	84%	0.0%	rice ✓
#7	rice	rice ✓	80%	0.0%	rice ✓
#8	rice	rice ✓	73%	0.0%	rice ✓
#9	rice	rice ✓	86%	0.0%	rice ✓
#10	idly	idly ✓	84%	1.4%	rice ✗
#11	idly	idly ✓	82%	0.5%	rice ✗
#12	idly	idly ✓	85%	0.5%	rice ✗
#13	idly	idly ✓	80%	1.8%	rice ✗
#14	idly	idly ✓	91%	2.6%	rice ✗
#15	idly	idly ✓	87%	7.2%	rice ✗
#16	idly	idly ✓	57%	0.3%	rice ✗
#17	idly	idly ✓	72%	12.3%	rice ✗
#18	idly	idly ✓	80%	0.6%	rice ✗
#19	idly	idly ✓	89%	0.8%	rice ✗

Table A.1 Leave-one-out and zero-shot result for each of the 20 real photographs

C. HOW TO RUN

```
pip install -r requirements.txt
python app.py
# open http://127.0.0.1:7860
```

1. In **Teach**, type a class name, upload 5–20 photos and press **Add examples**.
2. Repeat for at least one more class.
3. Press **Train model**; the status line reports the image count and the training accuracy.
4. Open **Classify** and upload, paste or take a webcam photo.
5. Use **Zero-shot** at any time to classify against typed labels.